

## AdvR 09 – Function Factories & Operators

## Function Factories

# What Is a Function Factory?

A **function factory** makes functions. The manufactured function's enclosing environment is an execution environment of the factory:

```
power1 <- function(exp) {  
  function(x) x ^ exp  
}  
  
square <- power1(2)  
cube   <- power1(3)  
c(square(3), cube(3))
```

```
#> [1]  9 27
```

Building blocks: first-class functions + function environments + execution environments.

Each call to the factory creates a **separate** execution environment:

```
fn_env(square)$exp
```

```
#> [1] 2
```

```
fn_env(cube)$exp
```

```
#> [1] 3
```

`square` and `cube` have identical bodies but different enclosing environments — that's what makes them behave differently.

# Forcing Evaluation

Lazy evaluation causes a subtle bug:

```
x <- 2
square <- power1(x)
x <- 3
square(2)    # returns 8, not 4!
```

```
#> [1] 8
```

Fix: **force** evaluation in the factory:

```
power2 <- function(exp) {
  force(exp)
  function(x) x ^ exp
}
x <- 2; square <- power2(x); x <- 3
square(2)    # correct: 4
```

```
#> [1] 4
```

**Rule:** always `force()` arguments used only by the manufactured function.

# Stateful Functions

Factories + `<<-` = functions that **remember** across calls:

```
new_counter <- function() {  
  i <- 0  
  function() { i <<- i + 1; i }  
}  
counter_one <- new_counter()  
counter_two <- new_counter()  
c(counter_one(), counter_one(), counter_two())
```

```
#> [1] 1 2 1
```

Each manufactured function has its own enclosing environment, so counters are **independent**.

**Tip:** when state grows complex, switch to R6.

## Garbage Collection in Factories

Manufactured functions retain their enclosing environment. Large temp objects persist:

```
f1 <- function(n) {  
  x <- runif(n); m <- mean(x)  
  function() m  
}  
f2 <- function(n) {  
  x <- runif(n); m <- mean(x); rm(x)  
  function() m  
}  
c(lobstr::obj_size(f1(1e6)),  
  lobstr::obj_size(f2(1e6)))
```

```
#> * 8.00 MB
```

```
#> * 504 B
```

**Rule:** `rm()` large temp objects before returning the closure.

## Exercises (Factory Fundamentals)

- 1 Why is `force(x)` better than just writing `x` in a factory?
- 2 Create `pick(i)` that returns a function subsetting `x` with `i`:

```
pick(1)(x)          # equivalent to x[[1]]  
lapply(mtcars, pick(5))
```

- 3 What happens without a closure? Predict, then verify:

```
i <- 0  
new_counter2 <- function() { i <- i + 1; i }
```

- 4 What if you use `<-` instead of `<<-`?

```
new_counter3 <- function() {  
  i <- 0  
  function() { i <- i + 1; i }  
}
```



- 1 `force(x)` clearly signals intent — you're forcing evaluation, not accidentally typing a bare `x`. Both do the same thing, but `force()` is self-documenting.

2

```
pick <- function(i) {  
  force(i)  
  function(x) x[[i]]  
}  
identical(mtcars[[5]][[1]], pick(5)(mtcars[[1]]))
```

```
#> [1] FALSE
```

## Solutions (Factory Fundamentals) — 3–4

- ③ Without a closure, `i` lives in the **global environment** — it can be overwritten or interfere with other code.  
No encapsulation:

```
i <- 0
new_counter2 <- function() { i <- i + 1; i }
c(new_counter2(), new_counter2())
```

```
#> [1] 1 2
```

```
i # global i is modified
```

```
#> [1] 2
```

- ④ With `<-` instead of `<<-`, the counter **always returns 1** — each call creates a fresh local `i` and never modifies the enclosing `i`:

```
new_counter3 <- function() {
  i <- 0
  function() { i <- i + 1; i }
}
c3 <- new_counter3()
c(c3(), c3(), c3())
```

```
#> [1] 1 1 1
```

## Factory Applications

## ggplot2 Labelling Factories

The `scales` package uses factories for axis label formatters:

```
y <- c(12345, 123456, 1234567)
comma_format()(y)
```

```
#> [1] "12,345"    "123,456"    "1,234,567"
```

```
number_format(scale = 1e-3, suffix = " K")(y)
```

```
#> [1] "12 K"      "123 K"     "1 235 K"
```

Works with ggplot2's `labels` argument:

```
ggplot(df, aes(x, y)) +
  scale_y_continuous(labels = comma_format())
```

**Pattern:** factory configures formatting rules → manufactured function applies them.

## Box-Cox Transformation

Re-formulating as a factory makes it easy to explore:

```
boxcox2 <- function(lambda) {  
  if (lambda == 0) function(x) log(x)  
  else function(x) (x ^ lambda - 1) / lambda  
}  
c(boxcox2(1)(4), boxcox2(0)(4))
```

```
#> [1] 3.000000 1.386294
```

The factory fixes `lambda`; the manufactured function takes `x`. Clean separation of roles.

# Maximum Likelihood with Factories

Factory precomputes constants, speeding up optimisation:

```
ll_poisson <- function(x) {  
  n <- length(x); sum_x <- sum(x)  
  c <- sum(lfactorial(x))  
  function(lambda) {  
    log(lambda) * sum_x - n * lambda - c  
  }  
}  
  
x1 <- c(41, 30, 31, 38, 29, 24, 30, 29, 31, 38)  
optimise(ll_poisson(x1), c(0, 100), maximum = TRUE)
```

```
#> $maximum  
#> [1] 32.09999  
#>  
#> $objective  
#> [1] -30.26755  
#> ... [output truncated]
```

**Pattern:** data → factory → likelihood function → `optimise()`.

Factory fits model once; manufactured function resamples:

```
boot_model <- function(df, formula) {  
  mod <- lm(formula, data = df)  
  fitted <- unname(fitted(mod))  
  resid <- unname(resid(mod))  
  rm(mod)  
  function() fitted + sample(resid)  
}  
boot_mtcars <- boot_model(mtcars, mpg ~ wt)  
head(boot_mtcars())
```

```
#> [1] 18.73946 21.00000 23.00293 24.26639 17.80000
```

```
#> [6] 19.99431
```

No need to `force()` — `df` and `formula` are used by `lm()` before the closure is returned.

## Exercises (Factory Applications)

- ❶ Why might you formulate Box-Cox with  $x$  fixed and  $\lambda$  varying?

```
boxcox3 <- function(x) {  
  function(lambda) {  
    if (lambda == 0) log(x)  
    else (x ^ lambda - 1) / lambda  
  }  
}
```

- ❷ Why doesn't `boot_permute()` store a copy of the data?

```
boot_permute <- function(df, var) {  
  n <- nrow(df); force(var)  
  function() {  
    col <- df[[var]]  
    col[sample(n, replace = TRUE)]  
  }  
}
```



## Solutions (Factory Applications)

- ① Fixing `x` lets you try many `lambda` values on the **same data** — useful for finding the best transformation. You can give it a descriptive name like `boxcox_airpassengers <- boxcox3(AirPassengers)`.
- ② R's **copy-on-modify** semantics mean `df` inside the closure is just a name pointing to the same object in memory — no copy is made unless `df` is modified (which it isn't):

```
boot_permute <- function(df, var) {  
  n <- nrow(df); force(var)  
  function() df[[var]][sample(n, replace = TRUE)]  
}  
bp <- boot_permute(mtcars, "mpg")  
lobstr::obj_sizes(mtcars, bp)
```

```
#> * 7.21 kB
```

```
#> * 1.51 kB
```

## Factories + Functionals

## Generating Function Families

Combine `map()` with a factory to create many functions at once:

```
names <- list(  
  square = 2, cube = 3,  
  root = 1/2, reciprocal = -1  
)  
funs <- purrr::map(names, power2)  
funs$root(64)
```

```
#> [1] 8
```

```
funs$reciprocal(4)
```

```
#> [1] 0.25
```

Access without `funs$` prefix:

```
with(funs, root(100))
```

```
#> [1] 10
```

1 Which is equivalent to `with(x, f(z))`?

- ☐ a `x$f(x$z)`
- ☐ b `f(x$z)`
- ☐ c `x$f(z)`
- ☐ d `f(z)`
- ☐ e It depends

- ❶ (e) **It depends.** With a data frame, `with()` evaluates in a temporary environment where columns are visible, so `f(z)` looks up `f` and `z` in that environment, then the calling environment. The answer depends on what `x` contains:

```
f <- mean; z <- 1
x <- list(f = mean, z = 1)
identical(with(x, f(z)), x$f(x$z))  # TRUE
```

```
#> [1] TRUE
```

```
identical(with(x, f(z)), f(x$z))    # TRUE
```

```
#> [1] TRUE
```

```
identical(with(x, f(z)), f(z))      # TRUE
```

```
#> [1] TRUE
```

All four are equivalent here because `f` and `z` in `x` match the global ones.

## Function Operators

# What Is a Function Operator?

A **function operator** takes a function as input and returns a modified function:

```
chatty <- function(f) {  
  force(f)  
  function(x, ...) {  
    message("Processing ", x)  
    f(x, ...)  
  }  
}  
  
s <- c(3, 2, 1)  
purrr::map_dbl(s, chatty\(x) x^2))
```

```
#> [1] 9 4 1
```

Function operators are function factories that take a **function** as input. Python calls them **decorators**.

## safely() — Capturing Errors

`purrr::safely()` wraps a function to return result + error:

```
x <- list(1, 10, "oops")
out <- purrr::map(x, purrr::safely(log))
str(out[[3]])
```

```
#> List of 2
#> $ result: NULL
#> $ error :List of 2
#> ... [output truncated]
```

Use `list_transpose()` to restructure:

```
out <- purrr::list_transpose(purrr::map(x, purrr::safely(log)))
ok <- purrr::map_lgl(out$error, is.null)
out$result[ok]
```

```
#> [[1]]
#> [1] 0
#>
#> ... [output truncated]
```

Related: `possibly()` (default on error), `quietly()` (captures messages/warnings).



## memoise() — Caching Results

`memoise::memoise()` remembers previous inputs → instant on repeat calls:

```
slow_fn <- function(x) { Sys.sleep(1); x * 10 }  
fast_fn <- memoise::memoise(slow_fn)  
system.time(fast_fn(1)) # ~1 second  
system.time(fast_fn(1)) # instant (cached)
```

**Classic example:** memoised Fibonacci is  $O(n)$  instead of exponential:

```
fib <- memoise::memoise(function(n) {  
  if (n < 2) return(n)  
  fib(n - 2) + fib(n - 1)  
})
```

**Warning:** only memoise **pure** functions (output depends solely on input).

## Building Your Own: `delay_by()` and `dot_every()`

```
delay_by <- function(f, amount) {  
  force(f); force(amount)  
  function(...) { Sys.sleep(amount); f(...) }  
}
```

```
dot_every <- function(f, n) {  
  force(f); force(n)  
  i <- 0  
  function(...) {  
    i <- i + 1  
    if (i %% n == 0) cat(".")  
    f(...)  
  }  
}
```

**Compose** them with the pipe:

```
walk2(urls, paths,  
  download.file %>% dot_every(10) %>% delay_by(0.1))
```

## Exercises (Function Operators)

- 1 What does `base::Vectorize()` do? When might you use it?
- 2 Weigh the pros and cons of:

```
download.file %>% dot_every(10) %>% delay_by(0.1)
# versus
download.file %>% delay_by(0.1) %>% dot_every(10)
```

- 3 Should you memoise `download.file()`? Why or why not?

- ❶ `Vectorize()` wraps a non-vectorised function so it iterates over vector arguments (via `mapply()`). It does **not** speed up the function — it just changes the input interface. Use when you need to pass vector arguments to a scalar function.
- ❷ Both take the same total time. **First:** dot is printed, then delay occurs before download starts. **Second:** delay occurs, then download starts, then dot is printed after 10th download completes. The difference is timing of the dot relative to the delay.
- ❸ Generally **no**. Files at a URL may change, so memoising would serve stale content. Also, downloaded files can be large, consuming lots of memory. Only memoise if files are small, immutable, and downloaded repeatedly.